Criando seu primeiro bot para o telegram utilizando python

Ministrante: Bernardo de Castro Monteiro Franco Gomes



Sobre mim:

Graduando em Ciência da Computação - UFSJ

- Possuo 20 anos
- Dou aulas de Desenvolvimento Full-Stack
- Desenvolvo projetos, minicursos e palestras para feiras de tecnologia e para faculdade
    - Projeto: Valyra e Orienta.IA (Open-Source)
    - Tutoriais: PySul 2025 e 2026; Caipyra 2026; PyNe 2026;
    - Palestras: UFSJ SECOMP 2025; PyNe 2026

Divisão da Palestra

INTRODUÇÃO

0.0 - O que é o telegram?

0.1 - Criando uma conta no telegram

BLOCO 1

1.1 - O que é um bot?

1.2 - Bot API e MtProto

1.3 - Criando o bot no BotFather

1.4 - Gerando keys e hashs de desenvolvimento

1.5 - Preparando o ambiente

Divisão da Palestra

BLOCO 2
2.1 - Estrutura mínima
2.2 - Primeiro comando
2.3 - Problema do contexto
2.4 - Outros comandos
2.5 - Entendendo async e await

BLOCO 3
3.1 - Mensagens de texto
3.2 - Botões
3.3 - Callback Query

Divisão da Palestra

BLOCO 4

4.1 - Consumindo uma API

4.2 - Variáveis de ambiente

4.3 - Próximos passos

BLOCO 5

5.1 - Por que existem várias bibliotecas?

5.2 - Telethon

5.3 - Quando escolher cada uma?

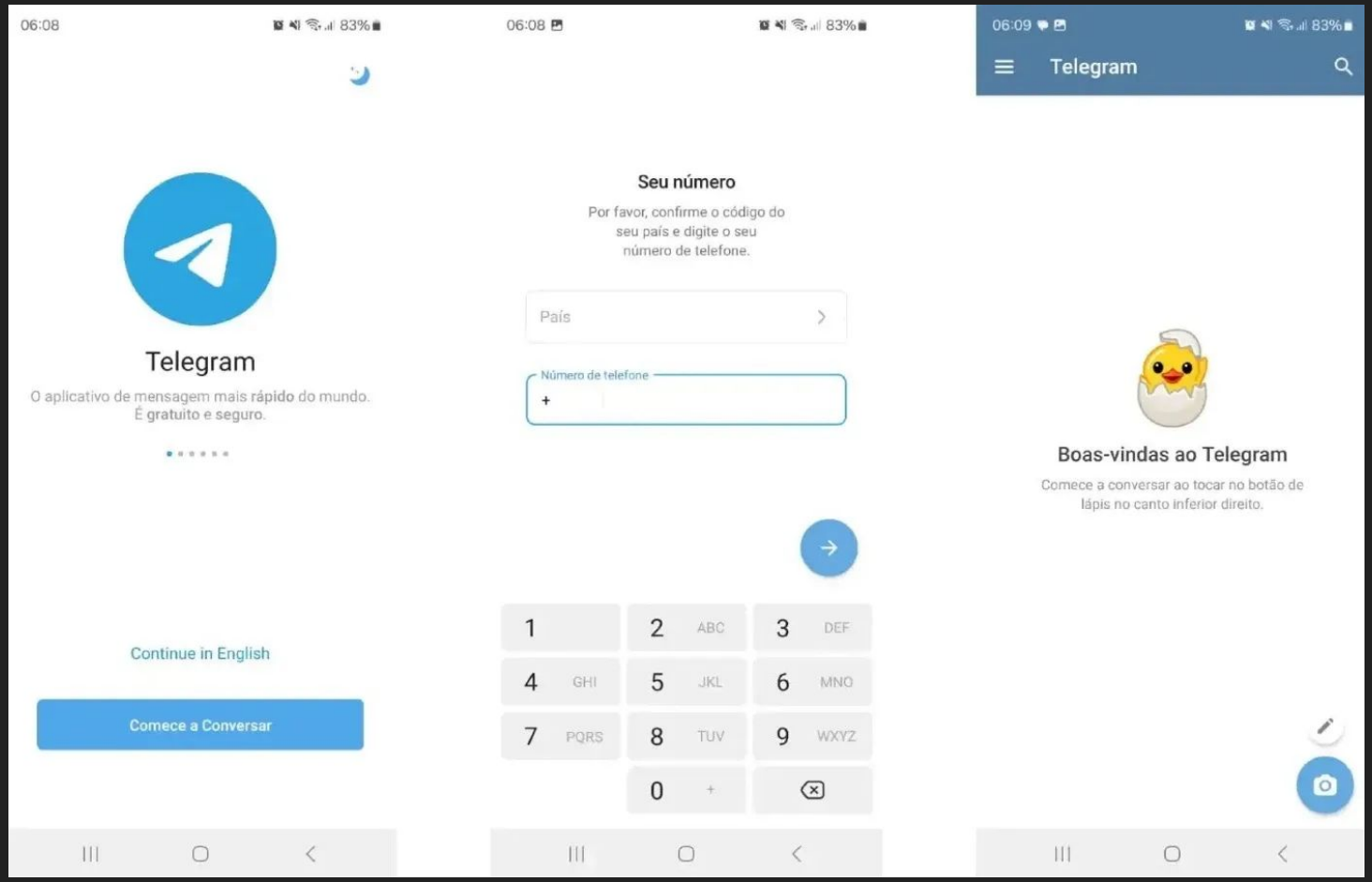
5.4 - Desafio prático

Introdução | O que é o telegram?

O Telegram é um aplicativo de mensagens com foco em velocidade e segurança, é super-rápido, simples e gratuito. Você pode usar o Telegram em todos os seus dispositivos ao mesmo tempo — suas mensagens são sincronizadas perfeitamente em qualquer quantidade de seus telefones, tablets ou computadores.



Introdução | Criando uma conta no telegram



B1 | O QUE É UM BOT?

Bot de Atendimento
Bot Educacional
Bot de Notícias
Bot Financeiro
Bot para Comunidades
Bot de Monitoramento
Bot integrado com IA
Bot de Vendas

O que eles têm em comum?
São capazes de interagir automaticamente com usuários

B1 | BotApi x MtProto

Pensada para criar bots

Fluxo de informações simples: Python -> Bot API -> Telegram

- Mais simples
- Limitado
- HTTP

É um protocolo utilizado pelo ecossistema do Telegram para comunicação mais direta com sua infraestrutura.

- Mais completo
- Protocolo próprio
- Poder total

B1 | BotApi x MtProto

Bot API é como utilizar uma porta de entrada já preparada para construir bots. MTProto permite trabalhar em uma camada mais baixa e oferece outras possibilidades.

B1 | BotApi x MtProto

Bot API é como utilizar uma porta de entrada já preparada para construir bots. MTProto permite trabalhar em uma camada mais baixa e oferece outras possibilidades.

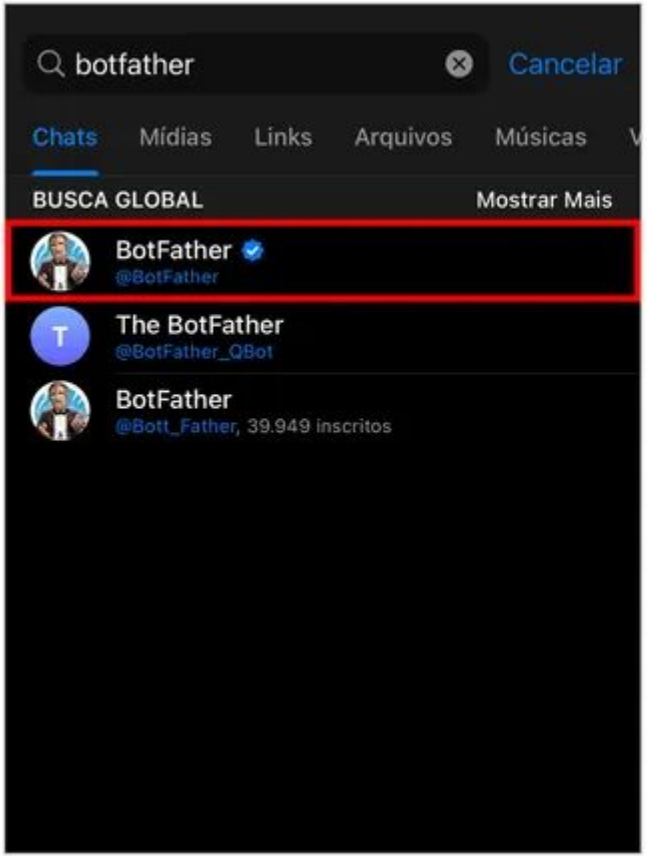
Mais baixo nível não significa automaticamente que é melhor

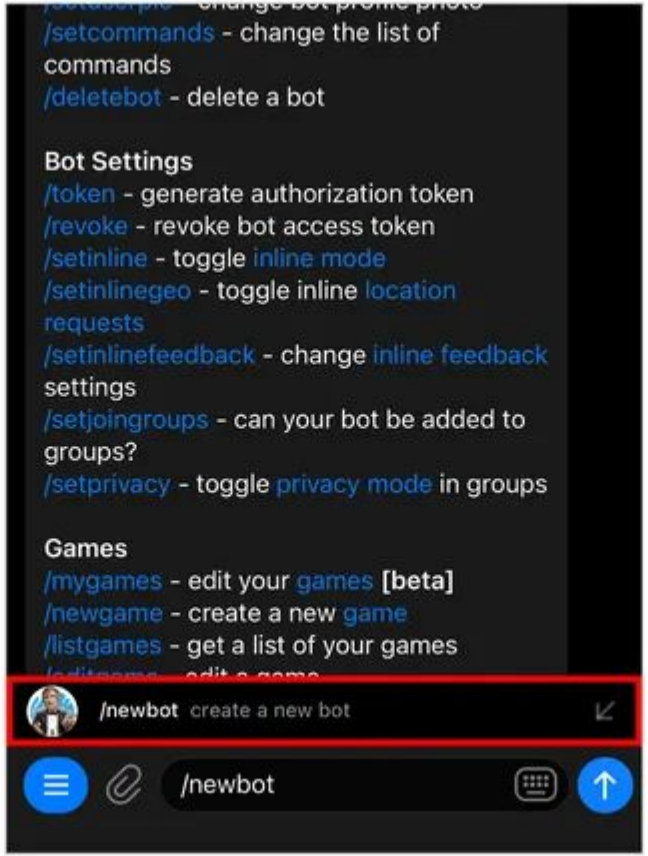
B1 | MtProto

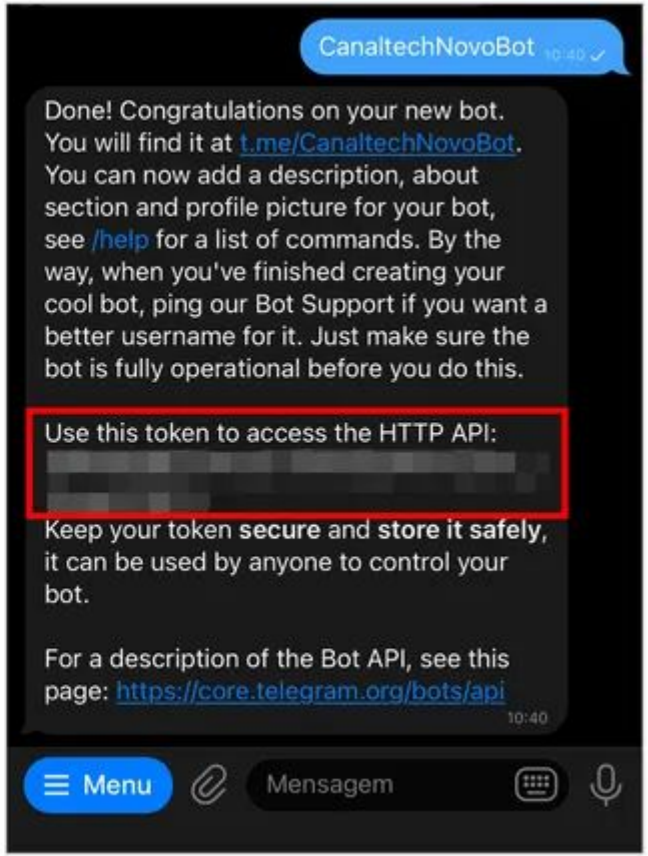
Protocolo de criptografia para garantir segurança, privacidade e alta velocidade nas mensagens, tanto em conversas na nuvem quanto em chats secretos. Utiliza de algoritmos modernos (como AES-256) para proteger a troca de dados.

São servidores intermediários especiais feitos sob medida para o aplicativo. Eles ofuscam o tráfego de rede para que governos ou provedores de internet não consigam bloquear ou identificar facilmente o uso do Telegram.

B1 | Criando o bot

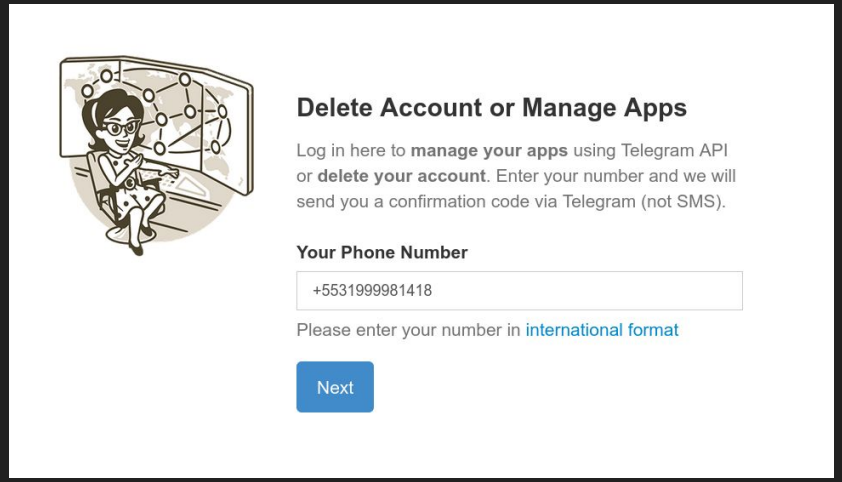






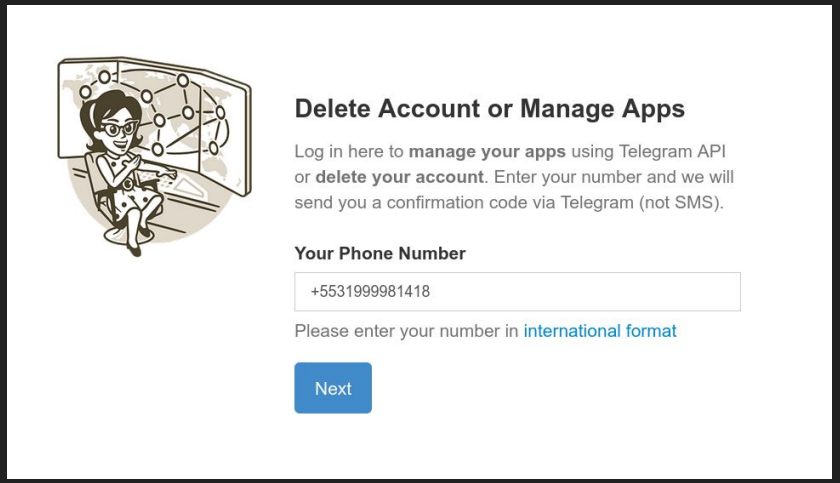
B1 | Keys e Hashs de desenvolvimento

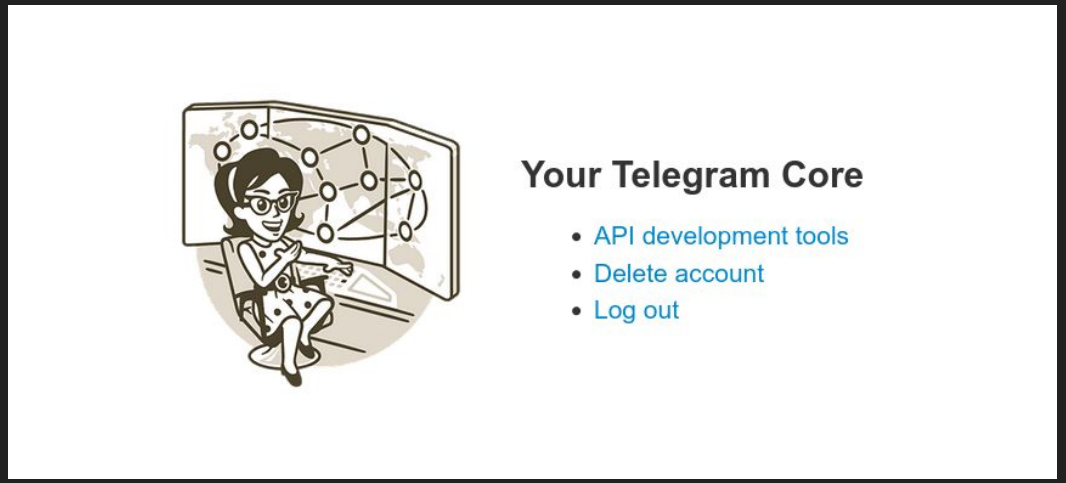
https://my.telegram.org/



B1 | Keys e Hashs de desenvolvimento

https://my.telegram.org/





B1 | Keys e Hashs de desenvolvimento

https://my.telegram.org/



Delete Account or Manage Apps

Log in here to **manage your apps** using Telegram API or **delete your account**. Enter your number and we will send you a confirmation code via Telegram (not SMS).

Your Phone Number

+5531999981418

Please enter

Next



Your Telegram Core

- API development tools
- Delete account
- Log out

App configuration

App api_id: [redacted]

App api_hash: [redacted]

App title: Aplicação Teste

Nome curto: Teste

alfanumérico, 5-32 caracteres

B1 | Preparando o ambiente

1º Biblioteca: Python-Telegram-Bot
2º Biblioteca: Telethon

https://github.com/bernardoxdev/Tutorial-PyBr26

Baixar a pasta “Template Código”
Ativar a venv
Link de ambas bibliotecas estão no README do repositório

B2 | Estrutura Mínima

```
meu-primeiro-bot/
│
├── .venv/
│
├── bot.py
│
└── requirements.txt
```

Vamos começar simples e depois vamos ir evoluindo nossa estrutura de projeto

B2 | Primeiro Comando

```
# Imports necessários para o primeiro comando:

from telegram import Update

from telegram.ext import (
    Application,
    CommandHandler,
    ContextTypes
)
```

B2 | Primeiro Comando

Update -> Representa uma atualização recebida pelo bot.

Application -> É o objeto principal que gerencia nossa aplicação.

CommandHandler -> Permite associar um comando a uma função.

ContextTypes -> Fornece o contexto daquela interação.

B2 | Primeiro Comando

```
async def start(
    update: Update,
    context: ContextTypes.DEFAULT_TYPE
):
    await update.message.reply_text("Olá! 👋")

app.add_handler(CommandHandler("start", start))
```

B2 | Primeiro Comando

```
app.add_handler(
    CommandHandler("start", start)
)
```

Utilizamos CommandHandler("start", start) e não CommandHandler("start", start())

“start” -> Estamos apenas passando a função
“start()” -> Estamos executando a função

```python
from telegram import Update
from telegram.ext import (
    Application,
    CommandHandler,
    ContextTypes
)

TOKEN = "SEU_TOKEN"

async def start(
    update: Update,
    context: ContextTypes.DEFAULT_TYPE
):
    await update.message.reply_text("Olá! Eu sou seu primeiro bot 🤖")

app = Application.builder().token(TOKEN).build()

app.add_handler(CommandHandler("start", start))

app.run_polling()
```

B2 | Problema do Contexto

Vamos supor que nosso bot está finalmente pronto para ser lançado. Precisamos entender como vamos lidar com usuários enviado múltiplas mensagens, ai que entra o *Problema do Contexto*.

Problema do Contexto é nada mais que tratar o fluxo de mensagens que chega para nosso bot

B2 | Problema do Contexto

Precisamos entender os seguintes tópicos:

- Cada mensagem é um novo evento, ou seja, chega de maneira independente
- O bot não “lembra” automaticamente do passado
- Fluxos interativos dependem de contexto

Para isso vamos criar um *Gerenciador de Estados*

B2 | Gerenciador de Estados

Precisamos entender que para gerenciar estados, vamos ter que criar uma “memória” para nosso bot, então vamos precisar de:

- Dicionário em memória
- ID dos usuários como chave
- Armazenar progresso

B2 | Fluxo de Interação

Vamos seguir o seguinte fluxo de interação:

- Pergunta x Resposta
- Validação de dados
- Próxima etapa

Vamos utilizar um fluxo de interação para “arrumar a casa”, agora o bot vai começar a “pensar”.

```python
from telegram import Update
from telegram.ext import (
    Application, CommandHandler, MessageHandler,
    ConversationHandler, ContextTypes, filters
)

NOME, = range(1)

async def iniciar(update: Update, context: ContextTypes.DEFAULT_TYPE):
    await update.message.reply_text("Qual é o seu nome?")
    return NOME

async def receber_nome(update: Update, context: ContextTypes.DEFAULT_TYPE):
    context.user_data["nome"] = update.message.text

    await update.message.reply_text(
        f"Olá, {context.user_data['nome']}!"
    )
    return ConversationHandler.END

app = Application.builder().token("SEU_TOKEN").build()

conversa = ConversationHandler(
    entry_points=[CommandHandler("start", iniciar)],
    states={
        NOME: [MessageHandler(filters.TEXT & ~filters.COMMAND, receber_nome)]
    },
    fallbacks=[]
)

app.add_handler(conversa)
app.run_polling()
```

B2 | Outros Comandos

Agora é hora para um desafio prático, a ideia é criar 2 comandos simples:

- /help -> Mostra todos os comandos que o bot possui
- /sobre -> Fala quem é o desenvolvedor do bot

B2 | Entendendo Async e Await

Imagine que o nosso bot faça os seguintes passos:

```
print("Vou consultar o banco...")
# consulta ao banco demora 3 segundos
print("Consulta terminou!")
print("Vou enviar a resposta...")
```

Se a consulta demora 3 segundos, o programa tem que ficar parado esperando

É como pedir um café e ficar parado olhando para o atendente durante 3 minutos. Com programação assíncrona, podemos dizer: "Enquanto estou esperando o café, posso fazer outra coisa."

B2 | Entendendo Async e Await

async -> Essa função pode trabalhar de forma assíncrona

await -> Preciso esperar essa operação terminar, mas enquanto isso outras tarefas assíncronas podem continuar

Exemplo:

```python
import asyncio

async def buscar_dados():
    print("Comecei a buscar")
    await asyncio.sleep(3)
    print("Terminei de buscar")
```

B3 | Handlers

Handler = Quando isso acontecer, faça aquilo

Ideia de fluxo do telegram:
Telegram -> Evento -> Handler -> Função Python

Exemplos:
/start -> CommandHandler
"Olá" -> MessageHandler
Clique em botão -> CallbackQueryHandler

B3 | Mensagem de texto

```
async def mensagem(update, context):
    await update.message.reply_text(
        "Recebi sua mensagem!"
    )
```

Criamos uma função para realizar uma ação quando recebermos uma mensagem, nesse caso é enviar uma mensagem falando “Recebi sua mensagem!”

B3 | Mensagem de Texto

```
app.add_handler(
    MessageHandler(
        filters.TEXT,
        mensagem
    )
)
```

Aqui registramos o handler da mensagem

B3 | Mensagem de Texto

```
async def mensagem(update, context):
    await update.message.reply_text("Recebi sua mensagem!")

app.add_handler(MessageHandler(filters.TEXT, mensagem))
```

B3 | Mensagem de texto

Assim nosso fluxo fica o seguinte:

👤 "Olá"
↓
MessageHandler
↓
mensagem()
↓
🤖 "Recebi sua mensagem!"

B3 | Filtros

Nem todas mensagem vamos tratar de todas as maneiras, para isso existem os filtros, podendo ser eles de vários tipos:

- filters.TEXT
- filters.PHOTO
- filters.VIDEO
- filters.Document.ALL
- filters.COMMAND

B3 | Menus

Para criar menus vamos precisar fazer alguns imports:

```
from telegram import InlineKeyboardButton
from telegram import InlineKeyboardMarkup
```

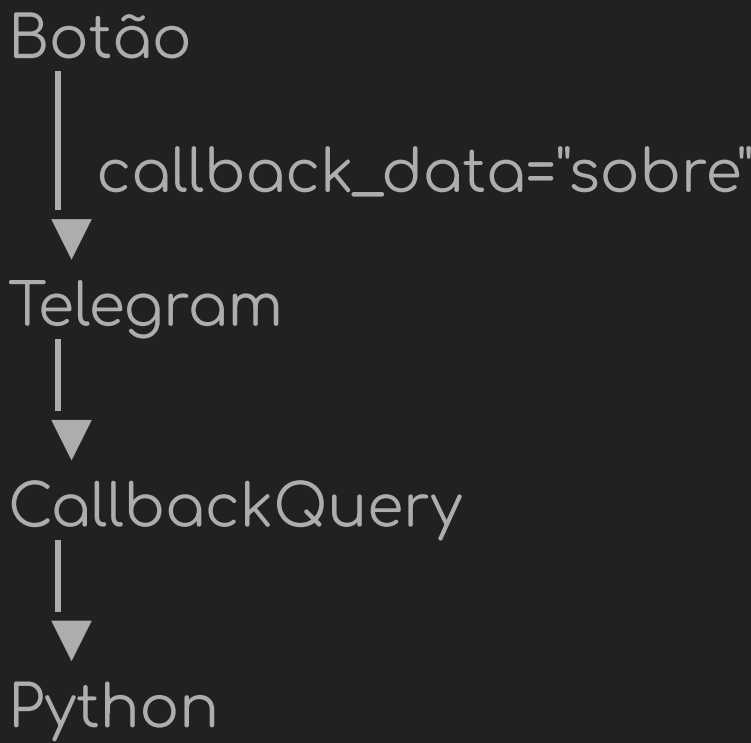
E partir deles podemos criar menus:

B3 | Menus

```
keyboard = [
    [
        InlineKeyboardButton(
            "📚 Sobre",
            callback_data="sobre"
        ),
        InlineKeyboardButton(
            "❓ Ajuda",
            callback_data="ajuda"
        )
    ],
    [
        InlineKeyboardButton(
            "📞 Contato",
            callback_data="contato"
        )
    ]
]

reply_markup = InlineKeyboardMarkup(keyboard)
```

B3 | Callback Query



B3 | Callback Query

Para conseguir tratar os Callbacks Querys vamos precisar utilizar o seguinte import:

```
from telegram.ext import CallbackQueryHandler
```

E utilizar como código base:

```
async def button(update, context):
    query = update.callback_query
    await query.answer()

app.add_handler(
    CallbackQueryHandler(button)
)
```

B3 | Callback Query

```
from telegram.ext import CallbackQueryHandler

async def button(update, context):
    query = update.callback_query
    await query.answer()

app.add_handler(CallbackQueryHandler(button))
```

B3 | Callback Query

Para descobrimos qual data estamos tratando, vamos precisar criar a seguinte variável dentro do nosso código:

```
async def button(update, context):
    query = update.callback_query
    acao = query.data
    await query.answer()
    await query.edit_message_text(f"Você clicou em {acao}!")
```

B3 | Desafio

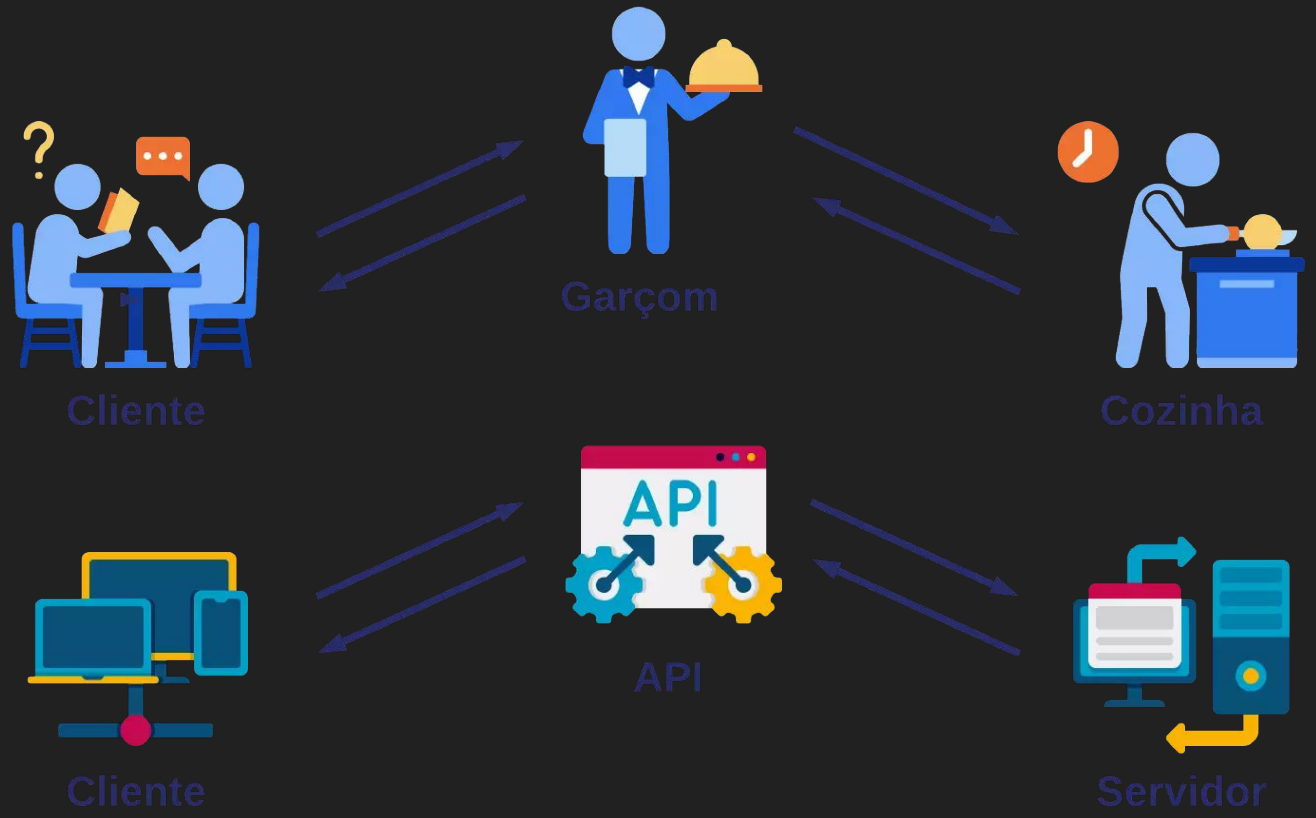
A ideia do nosso 1º desafio é criar um bot onde o usuário vai conseguir clicar em um botão para escolher um jogo, menu de exemplo:

🎮 Jogos

Escolha uma opção:

[🪨 Pedra/Papel/Tesoura]
[🎲 Dado]
[↩️ Voltar]

B4 | O que é uma API?



B4 | Consumindo uma API

Para consumir uma API precisamos primeiro entender o que o seguinte código faz:

```
import requests

response = requests.get(URL)
dados = response.json()
```

B4 | Consumindo uma API

requests.get()
↓
Faz a requisição

response
↓
Resposta da API

response.json()
↓
Transforma JSON em Python

B4 | Consumindo uma API

Como estamos fazendo um projeto de código assíncrono nossa melhor opção é usar a biblioteca *httpx* para não bloquear o event-loop

```
import httpx

async with httpx.AsyncClient() as client:
    response = await client.get(URL)

dados = response.json()
```

B4 | Falando sobre dados

Quando fazemos uma requisição - e utilizamos o response.json() - , o que o nosso código recebe é nada mais nada menos que um dicionário:

```
dados = {
    "cidade": "Belo Horizonte",
    "temperatura": 24,
    "condicao": "Ensolarado"
}
```

B4 | Tratamentos de Erros

Quando estamos trabalhando, podemos ter vários erros, como:

- API indisponível
- Timeout
- Resposta inválida
- Erro HTTP

B4 | Tratamentos de Erros

```
try:
    async with httpx.AsyncClient() as client:
        response = await client.get(URL)

    response.raise_for_status()

except httpx.HTTPError:
    await update.message.reply_text(
        "❌ Não consegui consultar o serviço."
    )
```

B4 | Variáveis de Ambiente

Uma das grandes práticas de segurança que possuímos hoje em dia é o uso de variáveis de ambiente, que fazem com que seja possível “esconder” dados sigilosos de nosso.

Por exemplo, sempre vamos querer evitar colocar em nosso código o nosso TOKEN e API_KEY

TOKEN = "123456789:ABC..."
API_KEY = "abc123..."

B4 | Variáveis de Ambiente

O que vamos fazer é criar um arquivo .env e colocar nossos dados dentro dele e nosso código vai ficar assim:

```
import os

TOKEN = os.getenv("BOT_TOKEN")
API_KEY = os.getenv("API_KEY")
```

B4.5 | Polling

Na técnica de polling o sistema fica sempre perguntando ao servidor: “Tem algum coisa nova?“. Exemplo:

Bot → Telegram: Tem mensagem nova?
Telegram → Bot: Não

Bot → Telegram: Tem mensagem nova?
Telegram → Bot: Não

Bot → Telegram: Tem mensagem nova?
Telegram → Bot: Sim! Mensagem: "Olá"

B4.5 | Polling

Vantagens:

- Mais simples de implementar.
- Não precisa necessariamente de um servidor acessível publicamente.
- Ótimo para desenvolvimento local.
- Fácil de entender para iniciantes.

Desvantagens:

- Faz requisições mesmo quando não há novidades.
- Pode consumir mais recursos.
- Existe um pequeno atraso dependendo do intervalo de consulta.

B4.5 | Webhook

Na técnica de webhook o sistema fala para o servidor: “Quando acontecer alguma coisa, me avise neste endereço.". Exemplo:

Usuário → Telegram: "Olá"

Telegram → Bot:
POST https://meubot.com/webhook

Bot → Telegram:
Recebi a mensagem!

Aqui o telegram inicia a comunicação quando existe um evento. É como deixar seu número de telefone para alguém te ligar quando precisar.

B4.5 | Webhook

Vantagens:

- Comunicação praticamente em tempo real.
- Não fica fazendo requisições desnecessárias.
- Mais eficiente para produção e alto volume.
- O servidor só recebe requisições quando há eventos.

Desvantagens:

- Exige uma aplicação acessível pela internet.
- Normalmente precisa configurar HTTPS.
- A configuração é um pouco mais complexa.
- É necessário lidar com disponibilidade e segurança do endpoint.

B5 | Por que existem várias bibliotecas?

O Telegram possui uma infraestrutura muito maior que a interface específica disponibilizada pela Bot API. Cada biblioteca que possuímos entra para fazer uma função diferente e em camadas diferentes do Telegram.

- python-telegram-bot -> Bots usando Bot API
- aiogram -> Bots assíncronos usando Bot API
- Pyrogram -> Cliente python que trabalha com a API do Telegram e Bot API
- Telethon -> Cliente python para a API do Telegram/MTProto

B5 | Telethon

Primeiro precisamos entender quando vamos utilizar o Telethon:

- Trabalhar como usuário
- Acessar conversas
- Trabalhar com grupos/canais
- Automação avançada
- Recursos da API do Telegram

Isso não significa que qualquer automação de usuário seja apropriada. Também existem regras e limitações do Telegram. A biblioteca não elimina essas regras.

B5 | Primeiro contato com o Telethon

```python
from telethon import TelegramClient, events

client = TelegramClient(
    "bot",
    API_ID,
    API_HASH
)

@client.on(
    events.NewMessage(pattern="/start")
)
async def start(event):
    await event.respond(
        "Olá! 🤖"
    )

client.start(
    bot_token=BOT_TOKEN
)

client.run_until_disconnected()
```

B5 | Quando escolher cada um?

Lembrando que isso não regras, apenas ideias:

Primeiro bot -> python-telegram-bot
Bot tradicional -> python-telegram-bot
Bot assíncrono -> aiogram
Trabalhar com MTProto -> Telethon

B5 | Desafio

Qual biblioteca vamos escolher em cada uma das situaçoes?

1 - Quero criar um bot de atendimento para uma loja?

2 - Quero criar um bot que consulta uma API de previsão do tempo?

3 - Preciso trabalhar com recursos da API do Telegram que vão além do modelo tradicional de Bot API?

B5 | Desafio

Qual biblioteca vamos escolher em cada uma das situaçoes?

1 - python-telegram-bot

2 - Quero criar um bot que consulta uma API de previsão do tempo?

3 - Preciso trabalhar com recursos da API do Telegram que vão além do modelo tradicional de Bot API?

B5 | Desafio

Qual biblioteca vamos escolher em cada uma das situaçoes?

1 - python-telegram-bot

2 - python-telegram-bot

3 - Preciso trabalhar com recursos da API do Telegram que vão além do modelo tradicional de Bot API?

B5 | Desafio

Qual biblioteca vamos escolher em cada uma das situaçoes?

1 - python-telegram-bot

2 - python-telegram-bot

3 - Telethon / Pyrogram (Descontinuada)

B5 | Desafio

Agora vamos criar um bot que faz o consumo da API do OpenMeteo: https://open-meteo.com/

Comandos obrigatórios: /start; /help; /clima; /sobre

O projeto deve conter: Algumas cidades como padrão onde o usuário vai escolher por meio de botões

Discussão e Q&A

Ficamos por aqui!
QUalquer dúvida:

(31) 99998-1418

bernardocmfgomes@gmail.com

@bernardocastro___
@bernardoxdev

